## Q1: Implementing Threads

### Design Decisions and Data Structures

The `clone()` and `join()` system calls implement thread creation and termination management. Unlike `fork()`, which creates a separate address space, `clone()` allows threads to share a page directory while maintaining separate process-table entries, kernel stacks, trap frames, and execution contexts.

The process structure includes an `isthread` flag and a `leader` pointer to distinguish threads from ordinary processes and identify their shared address-space owner. The `clone()` implementation initializes the thread's execution context using the supplied function, argument, and user stack. The `join()` implementation waits for an exited thread and reclaims its resources.

### Problems Faced and Resolution

The main challenges are initializing the user stack correctly, distinguishing threads from ordinary processes, and avoiding premature deallocation of shared memory. These issues require careful handling of thread creation, exit, joining, and address-space cleanup in the process-management code.

The implementation uses separate execution contexts and thread-group information to support these requirements. Correctness must be confirmed by testing thread creation, joining, and memory cleanup.

### Race-Condition Benchmark

The `threadtest.c` program creates three threads, each incrementing a shared counter 100,000 times without synchronization. Since incrementing involves reading, modifying, and writing a shared value, concurrent operations can cause lost updates.

| Metric | Q1: Without synchronization | Q2: With semaphore |
|---|---|---|
| Number of threads | 3 | 3 |
| Increments per thread | 100,000 | 100,000 |
| Expected final counter | 300,000 | 300,000 |
| Actual final counter | To be measured | To be measured |
| Lost updates | To be calculated | To be calculated |

Lost updates are calculated as:

**Lost updates = 300,000 − actual final counter**

The measured values will demonstrate the effect of synchronization on concurrent updates.

### 3. Q2: Process and Thread Synchronization Using Semaphores

#### Design Decisions and Data Structures

The kernel semaphore implementation maintains a fixed-size table of ten semaphores. Each entry contains a counter, an initialization flag, wakeup-ticket information, and a spinlock protecting its state.

The following system calls are implemented:

- `sem_init(id, value)` initializes a semaphore.
- `sem_wait(id)` decrements the counter and blocks when resources are unavailable.
- `sem_signal(id)` increments the counter and wakes waiting processes when necessary.

The implementation uses xv6's `sleep()` and `wakeup()` mechanisms. Per-semaphore spinlocks protect counter updates, while wakeup tickets are used to control which waiting operations may proceed.

The `synctest.c` program contains two tests. The first protects the shared counter using a binary semaphore. The second implements a bounded-buffer producer-consumer problem using `empty`, `full`, and `mutex` semaphores.

#### Problems Faced and Resolution

The main challenges include protecting shared data, avoiding lost wakeups, and coordinating the producer and consumer without buffer overflow or underflow.

The semaphore implementation uses spinlocks to protect semaphore state and a sleep/wakeup mechanism to block waiting processes. The producer-consumer implementation uses separate semaphores for available slots, occupied slots, and mutual exclusion.

#### Expected Benchmark Outcome

With three threads performing 100,000 increments each, the semaphore-protected counter should reach 300,000. The producer-consumer test should successfully process all 15 items while maintaining the bounded-buffer constraints.

The actual counter values, produced and consumed item counts, and any detected errors must be recorded after execution.

### 4. Q3: Deadlock Detection and Avoidance

#### Design Decisions and Data Structures

The user-space program `deadlock.c` simulates resource allocation using the following structures:

- **Allocation:** Resources currently held by each process.
- **Request:** Resources each process is currently waiting for.
- **Max:** Maximum resources each process may claim.
- **Available:** Resources currently available.

- **Need:** Remaining resources required to reach each process's maximum claim.

The Need matrix is calculated using:

**Need = Max − Allocation**

**Algorithms**

The deadlock-detection algorithm initializes `Work` to `Available`. It repeatedly searches for an unfinished process whose current request can be satisfied by `Work`. When such a process can finish, its allocated resources are returned to `Work`. Any processes that remain unfinished after no further progress can be made are identified by the detector.

The Banker's algorithm uses the Need matrix to determine whether a safe execution sequence exists. Resource requests are tentatively applied and the resulting state is checked. If the state is unsafe, the tentative allocation is rolled back.

The implementation also includes a pipe-based demonstration involving two processes that block while waiting on pipe operations.

**Deadlock Detection Results**

The following table should be completed using the actual program output.

| Test case | Test objective | Actual observation |
|---|---|---|
| Case 1 | Evaluate a safe resource-allocation state | To be recorded |
| Case 2 | Evaluate an unsafe resource-allocation state | To be recorded |
| Case 3 | Evaluate a resource request and resulting safety state | To be recorded |
| Pipe-based test | Demonstrate a process-level deadlock | To be recorded |

For each matrix-based test, the report should include the initial Available, Allocation, and Request or Max matrices, intermediate Work values, processes that can finish, and the final safe sequence or unfinished-process set.

An unsafe state means that the algorithm cannot find a safe completion sequence. It does not necessarily mean that a deadlock already exists. Actual deadlock detection examines current outstanding requests and resource dependencies.

**5. Q4: Priority-Based Preemptive CPU Scheduling**

**Design Decisions and Data Structures**

The scheduler is modified to select the runnable process with the smallest numerical priority. Each process stores its priority and waiting-tick counter. New processes receive a default priority of 10, and the permitted priority range is 1–20, where 1 is the highest priority.

The scheduler retains round-robin behavior among processes with equal priority. Timer interrupts provide preemption, while the aging mechanism improves the priority of processes that remain runnable but are not scheduled.

When a process waits for more than 30 ticks, its priority is improved by one level, subject to the minimum value of 1. The system calls `set_priority()` and `get_priority()` support priority modification and inspection. The additional `get_waitticks()`system call allows the test program to observe waiting time.

**Problems Faced and Resolution**

The main challenges are selecting the correct runnable process, preserving preemption, maintaining fair scheduling among processes with equal priorities, and preventing starvation of low-priority processes.

The implementation uses priority-based process selection and a waiting-tick counter to support aging. Its effectiveness must be verified by observing process priorities, waiting times, and completion times during execution.

**Scheduling Benchmark**

The `priotest.c` program creates four CPU-bound child processes with initial priorities 2, 6, 12, and 20. It records completion information and samples priority and waiting-tick values.

| Process | Initial priority | Completion time (ticks) | Final priority |
|---|---|---|---|
| P1 | 2 | To be measured | To be measured |
| P2 | 6 | To be measured | To be measured |
| P3 | 12 | To be measured | To be measured |
| P4 | 20 | To be measured | To be measured |

A scheduling graph should be plotted with **initial priority on the x-axis** and **completion time in ticks on the y-axis**. The graph and table should be based on actual measurements.

Since a smaller numerical priority represents a higher priority, the results can be used to assess whether high-priority processes generally complete sooner. The sampled priority values should also show whether aging improves the priority of processes that wait for extended periods.

**6. Conclusion**

This assignment combines four important operating-system concepts: shared-address-space threads, semaphore-based synchronization, resource-allocation safety and deadlock detection, and priority-based CPU scheduling with aging.

The thread tests demonstrate the potential for race conditions when shared data is accessed without synchronization. The semaphore tests evaluate mutual exclusion and producer-consumer coordination. The deadlock simulator evaluates safe and unsafe resource-allocation states, while the scheduling test examines priority selection and the effect of aging.

The final assessment should be supported by successful build results, actual test outputs, the Q1–Q2 benchmark comparison, deadlock detection results for all test cases, and the Q4 completion-time-versus-priority graph.

**Testing status:** Actual measurements and screenshots must be added after executing the submitted programs in xv6. No test result should be reported as successful until it has been verified.